AA 274A Homework 1

Colin Skinner, *Stanford University*
colin13@stanford.edu

1. A* Motion Planning & Path Smoothing

1.1.

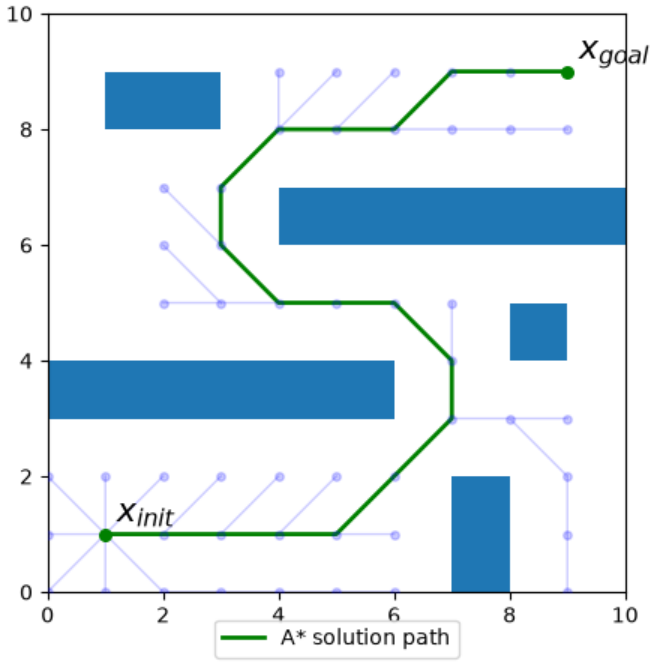


Figure 1: Basic Solution

1.2.

For the α parameter, I initially picked 0.05, but wanted to make it smoother to allow the robot to travel a shorter distance. I increased it to 0.5 and then decided to go up to the final value of 10.

When increasing the smoothing parameter α, as the trajectory becomes smoother, its length shortens. However, if it is too smooth, it might not be curved enough to avoid obstacles in an extremely cluttered environment. In this case, the obstacles are far enough from the A^* trajectory that it does not intersect no matter how much I increase it.

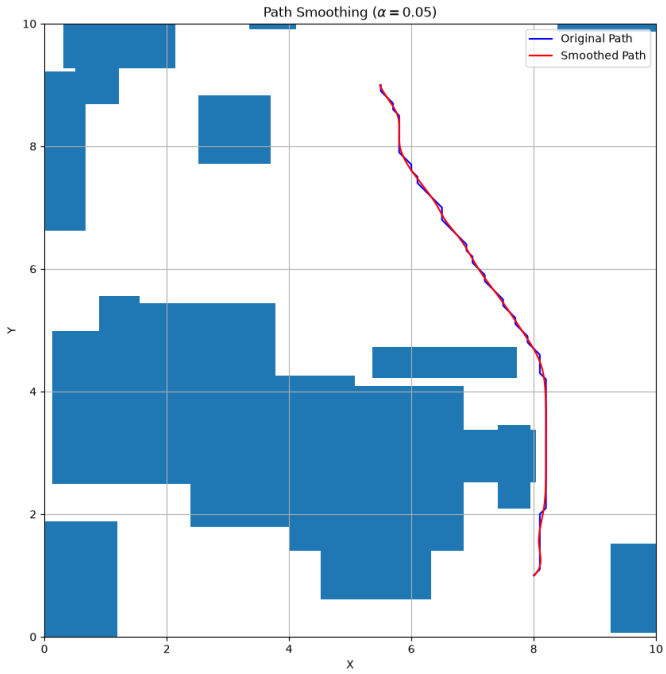


Figure 2: Trajectory with smoothing parameter of 0.05

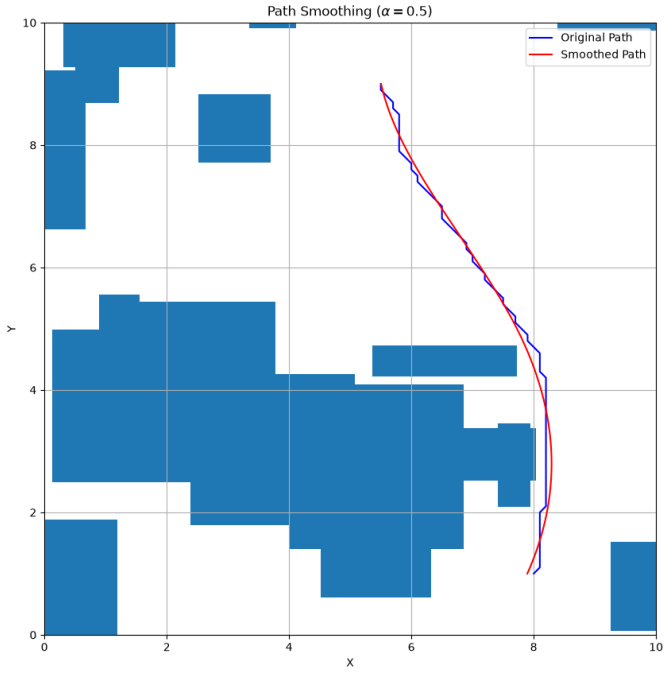


Figure 3: Trajectory with smoothing parameter of 0.5

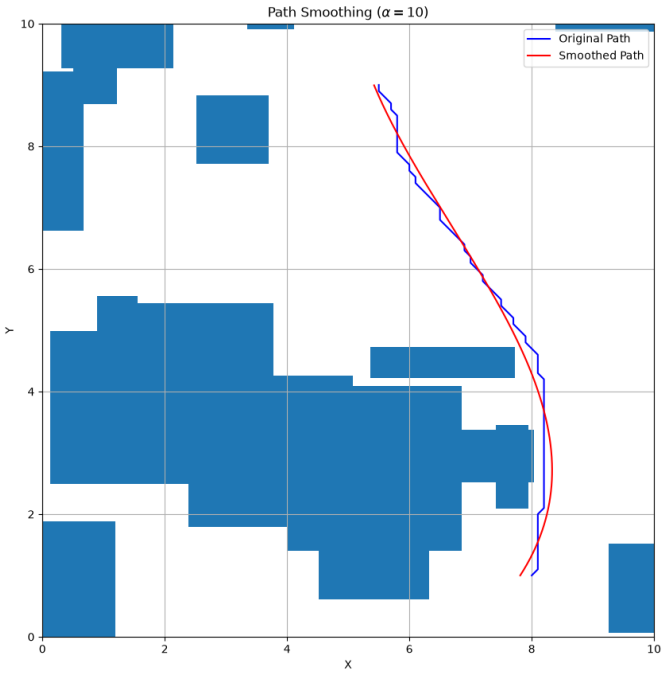


Figure 4: Trajectory with smoothing parameter of 10

1.3.

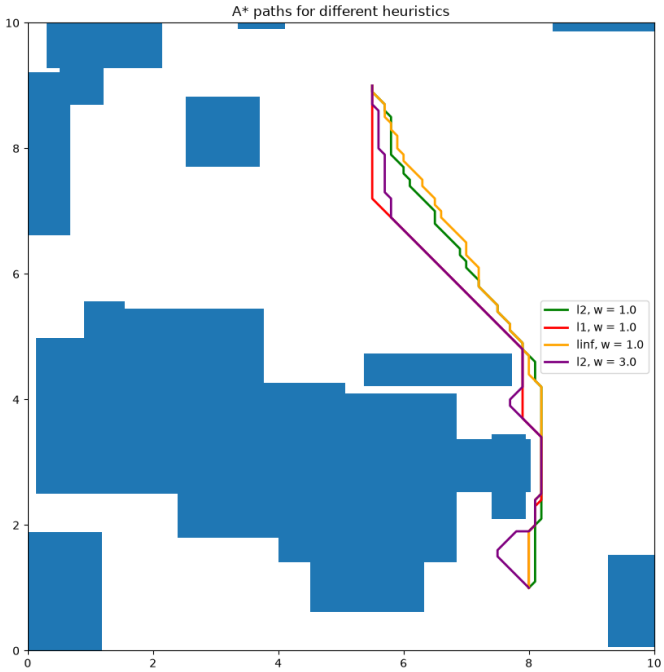


Figure 5: Weights

heuristic	w	path length	nodes expanded
12	1.0	9.201	1091

11	1.0	9.201	318
linf	1.0	9.201	1945
12	3.0	9.898	146

1.3.a

For the heuristics, the $\mathcal{L}^1$ trajectory is the only one that is not admissible. Since it totals the horizontal *and* vertical distances to the goal, if the robot is at any state that is not directly horizontal and vertical then the heuristic will overestimate the cost-to-go. For example, even in a state of $(6, 1)$, an optimal distance to the goal would have 5 straight lines and 1 diagonal to total 6.41, but the heuristic would add 6 and 1 to get 7. If, instead, diagonal moves were not possible, then it would be admissible since at most, it would be exactly equal to an optimal trajectory directly towards the goal.

For $\mathcal{L}^2$, this is the Euclidean distance, and this always takes the exact short path to the goal, so it always is equal to or less than the optimal trajectory. In the example above, the heuristic estimates 6.08, which is less than the optimal trajectory given the 8 possible moves at each state.

For $\mathcal{L}^\infty$, this takes the largest value out of both the horizontal and vertical directions. It is always smaller than a trajectory to the goal that does not lie directly horizontal or vertical to the goal, and equal when it does sit on those axes. This heuristic calculates 6 for the $(6, 1)$ state listed above.

1.3.b

In the order of nodes expanded: $\mathcal{L}^1 < \mathcal{L}^2 < \mathcal{L}^\infty$. If a heuristic is larger, then it is closer to the "true" cost and will tend to go in the direction of the optimal path. Therefore, it tends to reach the goal in relatively fewer nodes than a less informative heuristic.

The $\mathcal{L}^\infty$ norm underestimates the cost, and the $\mathcal{L}^1$ norm overestimates the cost, so both of those will explore or favor nodes that may not be close to the optimal direction (and may weigh them higher as well).

1.3.c

When the weighting is increased for the heuristic, the algorithm will tend to favor nodes that are more favored by the heuristic. In this case, sometimes the purple path ($w = 3$) will tend to drift off to the left side of the optimal path. As soon as it starts, it drifts off. Even though the optimal cost goes straight up, the upper left point has a smaller cost with the heuristic, so with the heuristic's increased weighting, it will favor the wrong node. This is shown in the path length, where it has a larger path length than the rest of the schemes.

For $\mathcal{L}^2$, increasing w from 1 to 3 increased the path length from 9.201 to 9.898, but decreased the number of expanded nodes from 1091 to 146. If the path length of 9.201 is the optimal one, then 9.898 is still, at most, less than 3 times that (optimality condition is satisfied)

$$9.898 \leq 3(9.201) = 27.603.$$

For $\mathcal{L}^1$ heuristic the other equation is

$$\|x\|_2 \leq \|x\|_1 \leq \sqrt{2}\|x\|_2.$$

which means it can overestimate the optimal cost-to-go by at most a factor of $\sqrt{2}$, giving a corresponding suboptimality bound. The result shows this too (easy to prove since both path lengths are the same for $\mathcal{L}^1$ and $\mathcal{L}^2$, so even though it is not admissible, it still produced the optimal path).

$$C_{\mathcal{L}^1} = 9.201 \leq \sqrt{2}(9.201) = 13.012.$$

Finally, a simple unit square demonstrates this, since the optimal path between diagonals is $\sqrt{2}$ but $\mathcal{L}^1$ has the heuristic at 2.

2. Rapidly-exploring Random Trees (RRT)

2.1.

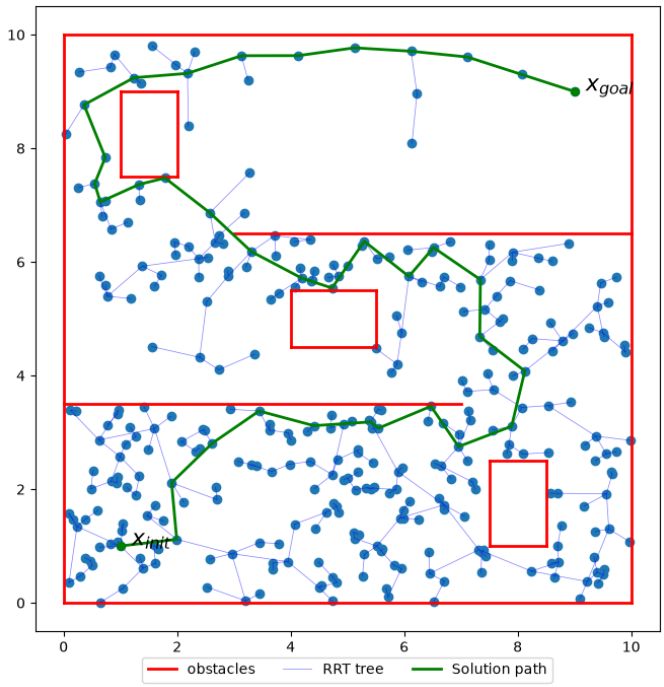


Figure 6: RRT solution with SEED $= 274$

2.2.

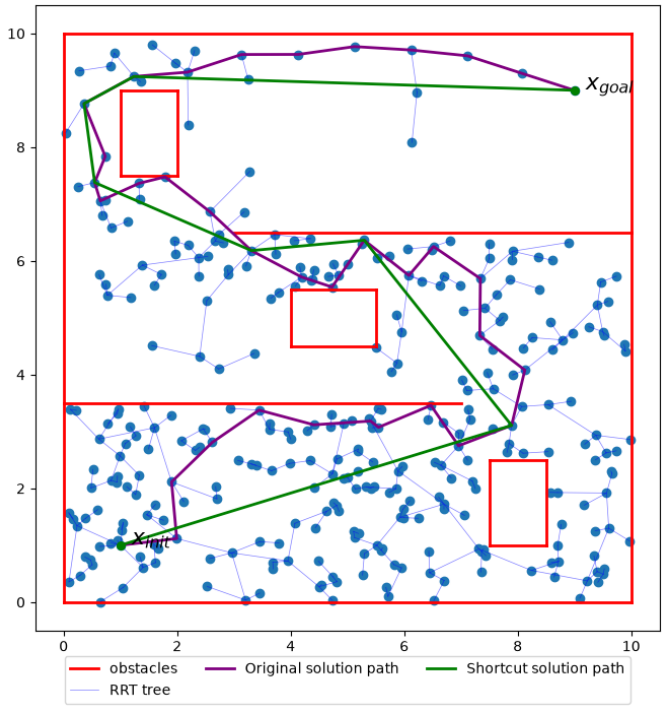


Figure 7: RRT shortcut solution with SEED $= 274$

2.3.

The table and plot for the goal-bias experiment are listed below.

goal bias	success rate	mean iterations
0.00	0.00	1999.0
0.05	1.00	390.9
0.20	1.00	403.2
0.50	1.00	614.1
0.90	0.20	1955.7

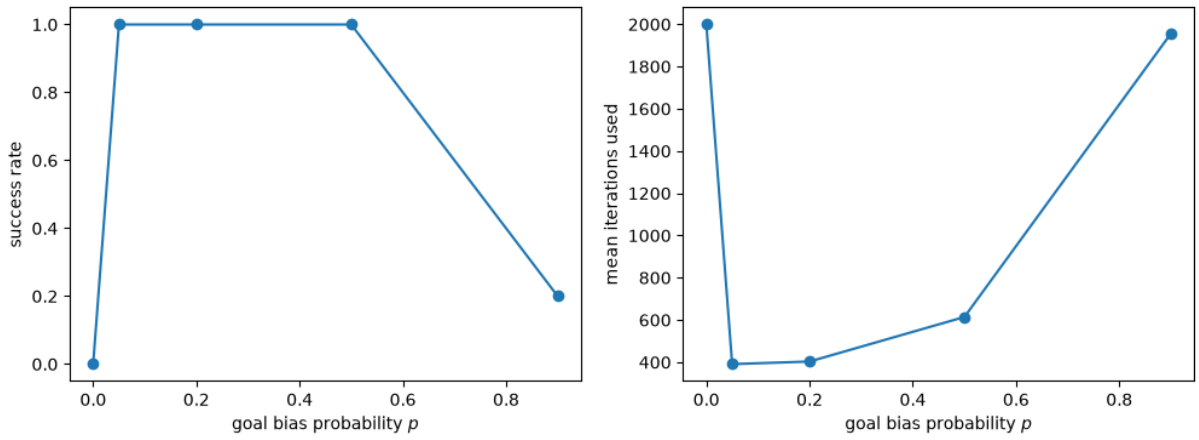


Figure 8: RRT results across varying goal biases

2.3.a

If p = 0, then the tree will never direct itself towards the goal. Even if it gets close, it still needs that small probability of having its next node be the goal in order to properly have its path go there (since it is in continuous space and not discrete space).

2.3.b

When a proposed path hits an obstacle, it just continues generating new points. If p is large, it will continually try to go towards the goal, and nearly every time, it will hit an obstacle and do nothing. When the simulation starts, the robot will hit the first wall immediately above it (after one or more steps) and then when it tries to go through, it doesn't do anything. Then it tries again, and since it has a very low chance of *not* do it (and doing a random node), it is essentially stuck or makes progress impossibly slow.

3. Heading Controller

3.1.

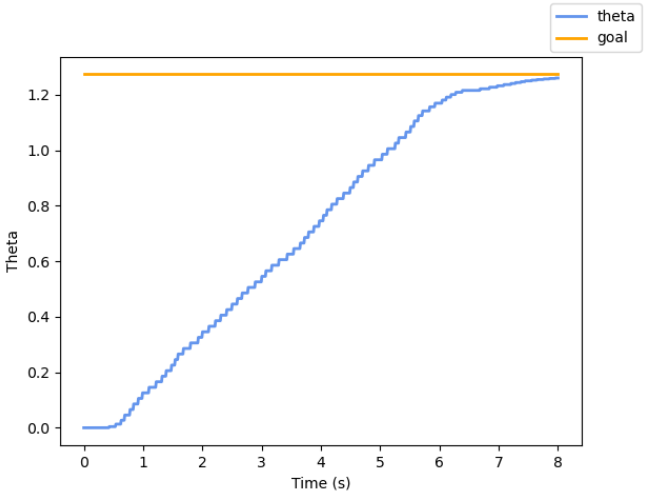


Figure 9: Commanded and goal angles

```
cskin@kevin:~/stanford/aa274a$ ros2 run autonomy_repo p3_plot.py
Commanding a goal angle, and saving a plot!
Please wait...
From initial angle of -1.7195134205394424e-12 to goal of 1.274
First 5% threshold crossing: 6.388 s
Successfully plotted to /home/cskin/stanford/aa274a/src/autonomy_repo/plots/p3_output.png
Close this node with CTRL+C!
```

Figure 10: Graph Output Text

3.2.

3.2.a

Deriving the time:

$$\begin{aligned} e(t) &= e(0)e^{-k_p t} \\ 0.05(1.274) &= 1.274e^{-2t} \\ t &= -\frac{\ln(0.05)}{2} = 1.498 \text{ s} \end{aligned}$$

Outputting the time at which the angle crosses the 5% threshold of error shows $\tau = 6.388s$.

3.2.b

For this condition to hold, we show

$$\begin{aligned} |1 - 0.1 * k_p| &< 1 \\ -1 < 1 - 0.1 * k_p &< 1 \\ -2 < -0.1 * k_p &< 0 \\ 0 < 0.1 * k_p &< 2 \end{aligned}$$

So we need this to be true for convergence

$$0 < k_p < 20$$

If $k_p < 10$, it would exponentially decay without any sign change, but $k_p > 10$ would cause the coefficient of e_k to be negative, which would cause alternating-sign error, but it would still be bound by an exponential decay envelope.

Then, if $k_p < 0$, error would just grow, and if $k_p > 20$ the error would still be sinusoidal, but it would grow exponentially as well

$$\begin{aligned} |w| &< w_{max} \\ -w_{max} < w &< w_{max} \\ -w_{max} < k_p * e &< w_{max} \\ -w_{max} < k_p * (\theta_{goal} - \theta) &< w_{max} \\ -w_{max}/k_p < \theta_{goal} - \theta &< w_{max}/k_p \\ -w_{max}/k_p < \theta_{goal} - \theta &< w_{max}/k_p \\ -w_{max}/k_p - \theta_{goal} < -theta &< w_{max}/k_p - \theta_{goal} \\ -w_{max}/k_p + \theta_{goal} < \theta &< w_{max}/k_p + \theta_{goal} \end{aligned}$$

And plugging in $w_{max} = 0.4$, $\theta_{goal} = 1.274$, and $k_p = 2$ we get:

$$1.074 < \theta < 1.474$$

So if θ is not within these bounds, the commanded ω will be larger than the threshold. Therefore, when the angle is smaller than the lower bound, the commanded angular velocity will saturate, so it will not travel as fast as it would if it were being commanded to the actual omega from the representative error.

For reasons why k_p cannot be arbitrarily large:

1. As stated in part b., it might cause the robot to overshoot or be too slow/sluggish
2. It might reach the physical limits of the motor itself. If this happens, it may try to command the motor to a higher ω than it is allowed to go, but the motor would saturate, and there would still be an error no matter how long the robot commands that ω. This could cause windup if there is an integral controller.